
Plantilla de Programación Competitiva - Python

Universidad de Bogotá Jorge Tadeo Lozano
Facultad de Ciencias Naturales e Ingeniería
Semillero de Programación Competitiva

Maratón Nacional ACIS REDIS 2026

Elaborado por: Nicolás Leitón

Bogotá D.C., 1 de octubre de 2026

Índice

1 Leer Input	1	4.6 Counter - Frecuencias	6	7.6 Kosaraju - Componentes	
1.1 n inicial y luego casos	1	4.7 Matrices	6	Fuertemente Conexas	11
1.2 Terminar cuando es 0 0	1	4.8 Heapq - Priority Queue	7	7.7 Bipartite Matching - Kuhn . . .	12
1.3 Input matriz nxm hasta EOF . . .	2	5 Arreglos numéricos	7	8 Geometría	12
1.4 Enumerar casos hasta EOF . . .	2	5.1 Two Sum - Doble puntero	7	8.1 Convex Hull - Monotonic Chain	12
2 Output Format	2	5.2 Sliding Window - Suma Mínima	7	8.2 Shoelace - Área de un Polígono	13
3 Macros	3	6 Strings	8	9 Matemáticas	13
3.1 Generar matriz	3	6.1 RollingHash	8	9.1 Aritmética básica	13
3.2 Memoización	3	6.2 Edit Distance	8	9.2 Potencias y logaritmos	13
3.3 Cambiar límite de recursión . . .	3	6.3 Z-Algorithm	9	9.3 Sumatorias	13
3.4 Optimizar input para leer		7 Grafos	9	9.4 Combinatoria	13
muchos datos	3	7.1 BFS - Camino más corto	9	9.5 Probabilidad	13
4 Estructuras de Datos	4	7.2 Dijkstra - Camino menos cos-		9.6 Geometría computacional	14
4.1 Bits - Operaciones y Bitmask . .	4	toso	10	9.7 Sucesiones	14
4.2 Collections - Deque	4	7.3 BFS - Para obtener distan-		9.8 Módulo y aritmética modular .	14
4.3 DefaultDict - Grafos	5	cias/profundidad	10	9.9 Binomio de Newton	14
4.4 Bisect - Búsqueda Binaria . . .	5	7.4 DFS - Recursivo	11	9.10 Contar dígitos	14
4.5 Sets - Conjuntos y operacio-		7.5 DFS - Iterativo	11	9.11 Conversiones útiles	14
nes en O(1)	5			9.12 Fórmulas de crecimiento	15
				9.13 Geometría	15

1 Leer Input

1.1 n inicial y luego casos

```
1 import sys
2 inputs = iter(sys.stdin.readlines())
3 TC = int(next(inputs))
4 for _ in range(TC):
5     print(sum(map(int, next(inputs).split())))
```

```
3
1 2
5 7
6 3
```

1.2 Terminar cuando es 0 0

```
1 import sys
2 for line in sys.stdin.readlines():
3     if line=='0 0\n': break
4     print(sum(map(int, line.split())))
```

```
1 2
5 7
6 3
0 0
1 1 <- no se procesa
```

1.3 Input matriz nxm hasta EOF

```
1
2 import sys
3 lines = sys.stdin.readlines()
4 i= 0
5
6 while i<len(lines):
7     # Leer dimensiones
8     n, m = map(int, lines[i].split())
9     i += 1
10
11     #Leer la matriz de tamaño n x m
12     matrix = [list(map(int, lines[i+j].split())) for j in range(n)]
13     i +=n
14 print(matrix)

2 3
1 2 3
4 5 6
--- out
[[1, 2, 3], [4, 5, 6]]
```

1.4 Enumerar casos hasta EOF

```
1
2 import sys
3 for c, line in enumerate(sys.stdin.readlines(), 1):
4     print("Case %s: %s\n" % (c, sum(map(int, line.split()))))

1 2
5 7
6 3
--- out
Case 1: 3
Case 2: 12
Case 3: 9
```

2 Output Format

```
1 # Mostrar 2 decimales
2 price = 45
3 print(f"The price is {price:.2f} dollars")
4 # The price is 45.00 dollars
5
6 # Usar ',' como separador de miles
7 age = 13800000000
8 print(f"The universe is {age:,} years old")
9 # The universe is 13,800,000,000 years old
10
11 # Convertir a porcentaje con 1 decimal (sin decimales usar 0)
12 score = 0.255
13 print(f"You scored {score:.1%}") # You scored 25.5%
14
15 # Convertir un numero a binario
16 num = 5
17 print(f"The binary version of {num} is {num:b}")
18 # The binary version of 5 is 101
```

3 Macros

3.1 Generar matriz

```
1 def crear_matriz(n, m, valor =0):
2     return [[valor for i in range(m)] for j in range(n)]
3     #j * m + i + 1   Reemplazar por valor para llenar la lista en orden de 1 a n*m
4
5 n, m = 3, 4
6 matriz= crear_matriz(n, m, 0)
7
8 """
9 0 0 0 0
10 0 0 0 0
11 0 0 0 0
12 """
```

3.2 Memoización

Guarda en memoria los resultados de las llamadas anteriores, para no calcularlas nuevamente

```
1 from functools import lru_cache
2
3 @lru_cache(maxsize=None)
4 def fibonacci(n):
5     (...)
6
7 #Si se necesita limpiar:
8 fibonacci.cache_clear()
```

3.3 Cambiar límite de recursión

```
1 import sys
2
3 sys.setrecursionlimit(10**6)
```

3.4 Optimizar input para leer muchos datos

Guarda todo como bytes, especialmente útil si las entradas son únicamente numéricas

```
1 import sys
2 input = sys.stdin.buffer.readline
```

4 Estructuras de Datos

Esta sección contiene las estructuras de datos más útiles de Python para programación competitiva.

4.1 Bits - Operaciones y Bitmask

```
1 # Operaciones basicas
2 a & b                # AND: bits prendidos en ambos
3 a | b                # OR: bits prendidos en alguno
4 a ^ b                # XOR: bits diferentes
5 ~a                  # NOT: invierte bits
6 a << n               # Desplazar n bits a la izquierda (* 2^n)
7 a >> n               # Desplazar n bits a la derecha (/ 2^n)
8
9 # Consultar bit n
10 (x >> n) & 1         # Retorna 0 o 1
11 x & (1 << n)         # True si el bit n esta prendido
12
13 # Modificar bit n
14 x |= (1 << n)        # Prender bit n
15 x &= ~(1 << n)        # Apagar bit n
16 x ^= (1 << n)        # Invertir bit n
17
18 # Recorrer bits
19 for i in range(k):    # Recorrer todos los bits [0, k-1]
20     bit = (x >> i) & 1
21
22 # Recorrer solo bits prendidos
23 while x:
24     bit = x & -x        # Bit prendido menos significativo
25     pos = bit.bit_length() - 1
26     x &= x - 1          # Eliminar ese bit
27
28 # Utilidades
29 x.bit_count()          # Cantidad de bits prendidos
30 x & -x                 # Obtener bit prendido menos significativo
31 x & (x - 1)            # Eliminar bit prendido menos significativo
32
33 # Potencia de 2
34 x > 0 and (x & (x - 1)) == 0
35
36 # Mascaras
37 (1 << n)                # Mascara con solo el bit n prendido
38 (1 << n) - 1           # Mascara con los primeros n bits prendidos
39 x & ((1 << n) - 1)      # Obtener los ultimos n bits
40
41 # Obtener bits [l, r]
42 (x >> l) & ((1 << (r-l+1)) - 1)
43
44 # Subconjuntos de una bitmask
45 sub = mask
46 while sub:
47     # procesar sub
48     sub = (sub - 1) & mask
49 # Tambien procesa el subconjunto vacio:
50 # while True:
51 #     ...
52 #     if sub == 0: break
53 #     sub = (sub - 1) & mask
54
55 # Ejemplo:
56 # x = 13 = 1101
57 # bit 0 = 1, bit 1 = 0, bit 2 = 1, bit 3 = 1
```

4.2 Collections - Deque

```
1 from collections import deque
2
3 dq = deque([1, 2, 3])
4 dq.append(4)        # [1, 2, 3, 4]
5 dq.appendleft(0)    # [0, 1, 2, 3, 4]
6 dq.popleft()        # -> 0
```

4.3 DefaultDict - Grafos

```
1 # Como un dict que crea automaticamente la llave cuando accedes a el
2 from collections import defaultdict
3 #Representar grafos con listas de adyacencia.
4 g = defaultdict(list)
5 n, m = 5, 6 #Nodos, aristas
6 edges = [(1,2), (1,3), (2,4), (3,4), (4,5), (5,1)]
7 for u, v in edges:
8     g[u].append(v)
9     g[v].append(u) # si es no dirigido
10
11 # Otro uso guardar elementos sin que se repitan
12 amigos = defaultdict(set)
13 amigos["Juan"].add("Ana")
14 amigos["Juan"].add("Pedro")
15 amigos["Juan"].add("Ana")
16 # 'Juan': {'Ana', 'Pedro'}
```

4.4 Bisect - Búsqueda Binaria

```
1 import bisect
2 #Siempre tiene que ser una lista ordenada
3 arr = [1, 3, 3, 5, 8]
4 # Busca en O(log(n))
5 L = bisect.bisect_left(arr, 1) # 0
6 R = bisect.bisect_right(arr, 3) # 3
7
8 #Cuantos elemento hay entre 1 y 3
9 print(R - L) # 3
10 #Insertar en orden
11 bisect.insort(arr, 4) #0(n)
12 # Parametros extra: (lo, hi) indice con los limites abajo y arriba para buscar
13 # (key) recibe una funcion como criterio de comparacion ej: key=lambda p: p[1] aqui busca el
    segundo elemento de cada lista
```

4.5 Sets - Conjuntos y operaciones en O(1)

```
1 # Crear
2 s = set() # Set vacio
3 s = {1, 2, 3} # Set con elementos
4 # Operaciones basicas (O(1) promedio)
5 s.add(4) # Agregar
6 s.remove(4) # Eliminar (KeyError si no existe)
7 s.discard(4) # Eliminar sin error
8
9 # Operaciones entre conjuntos
10 a | b # Union
11 a & b # Interseccion
12 a - b # Diferencia
13 a ^ b # Diferencia simetrica
14 # Relaciones
15 a <= b # a es subconjunto de b
16 a >= b # a es superconjunto de b
17
18 # Muy usado en BFS/DFS
19 visitados = set()
20 if nodo not in visitados:
21     visitados.add(nodo)
22 # Restriccion
23 # Solo admite elementos hashables (int, str, tuple...), No admite list, dict, set, etc.
```

4.6 Counter - Frecuencias

```
1 from collections import Counter
2 #Cuenta frecuencias mas rapido
3 cnt = Counter([1,2,2,3,3,3])
4 print(cnt[2]) # 2
5 print(cnt.most_common(1)) # [(3, 3)]
6
7 #Funciona con strings
8 conteo = Counter("hola buenas tardes")
9 print(conteo.most_common(2)) # [('a', 3), (' ', 2)]
```

4.7 Matrices

```
1 class matrix(): #Todas las operaciones tienen que ser con otro objeto de la misma clase
2     def __init__(self, values:list):
3         self.values = values
4         self.rows = len(values)
5         self.columns = len(values[0])
6
7
8     def __add__(self, other):
9         if self.rows != other.rows or self.columns != other.columns:
10             raise ValueError("Las columnas no son del mismo tamaño")
11
12         salida = [[0 for _ in range(self.columns)]
13                   for _ in range(self.rows)]
14         for i in range(self.rows):
15             for j in range(self.columns):
16                 salida[i][j] = self.values[i][j] + other.values[i][j]
17
18         return matrix(salida)
19
20
21     def __mul__(self, other):
22         if self.columns != other.rows:
23             raise ValueError("El numero de columnas de la primera matriz debe ser igual al numero
24                               de filas de la segunda.")
25
26         salida = [[0 for _ in range(other.columns)]
27                   for _ in range(self.rows)]
28         for i in range(self.rows):
29             for j in range(other.columns):
30                 for k in range(self.columns):
31                     salida[i][j] += self.values[i][k] * other.values[k][j]
32
33         return matrix(salida)
34
35     def __pow__(self, exponent): #binary exponentiation
36         if self.rows != self.columns:
37             raise ValueError("Solo las matrices cuadradas pueden elevarse a una potencia.")
38
39         result = matrix([[1 if i == j else 0 for j in range(self.rows)]
40                           for i in range(self.rows)])
41         base = self
42
43         while exponent > 0:
44             if exponent % 2 == 1:
45                 result = result * base
46
47             base = base * base
48             exponent //= 2
49
50         return result
51
52     def __str__(self):
53         return "\n".join(" ".join(map(str, fila)) for fila in self.values)
54     def __eq__(self, other): # Si se necesita comparar
55         return self.values == other.values
```

4.8 Heapq - Priority Queue

```
1 import heapq
2 # Sirve cuando necesitas sacar todo el tiempo el menor o mayor elemento
3 h = []
4 heapq.heappush(h, 3) # Insertar O(log n)
5 heapq.heappush(h, 1)
6 heapq.heappush(h, 5)
7 print(heapq.heappop(h)) # 1 (sacar el menor elemento) O(log n)
8 print(heapq.nlargest(2, h)) #Lista con los n elementos mas grandes (nsmallest para los menores)
9 heapq.merge(h1, h2) #Combina dos listas ordenadas devuelve un Iterador (se puede guardar como lista
    con list())
10
11 # Max heap:
12 h = [-3, 1, -5] #Invertir los numeros al insertar
13 heapq.heapify(h) #O(n)
14 print(-heapq.heappop(h)) # 5 (mayor)
```

5 Arreglos numéricos

5.1 Two Sum - Doble puntero

```
1 #La funcion intenta determinar si en el arreglo a (de tamaño n) existen dos numeros cuya suma sea
    igual a k.
2 def isPairSum(a, n, k):
3     global x, y
4     i, j = 0, n - 1
5     while i < j:
6         if a[i] + a[j] == k:
7             x, y = i, j
8             return 1
9         elif a[i] + a[j] > k:
10            j = j - 1
11        else:
12            i = i + 1
13    return 0
```

5.2 Sliding Window - Suma Mínima

```
1 # Halla la suma minima de un subarreglo de k elementos consecutivos en un array, y devuelve los
    indices donde empieza y termina dicho subarreglo.
2 def window(n:int, k:int, arr:list):
3
4     cursum = res_ind = 0
5     cursum = sum(x for x in arr[:k])
6     minsum = cursum
7
8     for i in range(k, n):
9         cursum = cursum + arr[i] - arr[i - k] #Deslizar ventana
10        if cursum < minsum:
11            minsum = cursum
12            res_ind = i - k + 1
13    return (res_ind, res_ind + k - 1)
14
15 n, k = map(int, input().split())
16 arr = list(map(int, input().split()))
17
18 print(*window(n, k, arr))
19
20 '''
21 #----INPUT----
22 7 3
23 3 7 90 20 10 50 40
24 #----OUTPUT----
25 3 5
26 '''
```

6 Strings

6.1 RollingHash

Sirve para comparar muchas subcadenas en $O(1)$, encontrar palindromos

```
1 class RollingHash:
2     MOD1 = 10**9 + 7
3     MOD2 = 10**9 + 9
4     BASE1 = 137
5     BASE2 = 277
6
7     def __init__(self, s: str):
8         self.s = s
9         n = len(s)
10
11         self.pref1 = [0] * (n + 1)
12         self.pref2 = [0] * (n + 1)
13         self.pot1 = [1] * (n + 1)
14         self.pot2 = [1] * (n + 1)
15
16         for i in range(n):
17             valor = ord(s[i]) # - ord('a') + 1 #Pequena optimizacion restar el caracter mas
18                             #pequeno de la restriccion
19
20             # Hash 1
21             self.pot1[i + 1] = (self.pot1[i] * self.BASE1) % self.MOD1
22             self.pref1[i + 1] = (self.pref1[i] * self.BASE1 + valor) % self.MOD1
23             # Hash 2
24             self.pot2[i + 1] = (self.pot2[i] * self.BASE2) % self.MOD2
25             self.pref2[i + 1] = (self.pref2[i] * self.BASE2 + valor) % self.MOD2
26
27         def hash(self, l: int, r: int) -> tuple[int, int]:
28             #Hash de s[l:r] (0-indexed, 'r' exclusivo)
29             h1 = (self.pref1[r] - self.pref1[l] * self.pot1[r - l]) % self.MOD1
30             h2 = (self.pref2[r] - self.pref2[l] * self.pot2[r - l]) % self.MOD2
31             return (h1, h2)
```

6.2 Edit Distance

Calcula el mínimo de operaciones a realizar sobre s1 para que sea igual s2 (siendo las operaciones quitar, insertar e cambiar caracter)

```
1 s1 = "goldens"
2 s2 = "glides"
3 n, m = len(s1), len(s2)
4
5 ed = [ [n - i if j == m else (m - j if i == n else 0) for j in range(m + 1)]
6         for i in range(n + 1)]
7
8 for i in range(n-1, -1, -1):
9     for j in range(m-1, -1, -1):
10         if s1[i]==s2[j]:
11             ed[i][j] = ed[i+1][j+1]
12
13         else:
14             ed[i][j] = min(ed[i+1][j], ed[i][j+1], ed[i+1][j+1]) + 1
```

6.3 Z-Algorithm

Calcula para cada posición i la longitud del prefijo del string que coincide con el substring que empieza en i . Encontrar todas las ocurrencias del prefijo. Complejidad de $O(n+m)$ o `text+pattern`

```

1 def z_function(s: str) -> list[int]:
2     n = len(s)
3     z = [0] * n
4     l, r = 0, 0
5
6     for i in range(1, n):
7         if i <= r:
8             z[i] = min(r - i + 1, z[i - 1])
9             while i + z[i] < n and s[z[i]] == s[i + z[i]]:
10                 z[i] += 1
11             if i + z[i] - 1 > r:
12                 l = i
13                 r = i + z[i] - 1
14
15     return z
16
17
18 def search_pattern(pattern: str, text: str) -> list[int]:
19     #Retorna todos los indices de inicio en 'text' donde aparece 'pattern'.
20     concat = pattern + "$" + text # '$' es un caracter que no esta en las cadenas
21     z = z_function(concat)
22     p_len = len(pattern)
23     matches = []
24
25     for i in range(p_len + 1, len(concat)):
26         if z[i] == p_len:
27             # Convertimos el indice de la cadena concatenada al indice original en 'text'
28             matches.append(i - (p_len + 1))
29
30     return matches
31
32 # --- EJEMPLO DE USO ---
33 text = "ababcababa"
34 pattern = "aba"
35 print(search_pattern(pattern, text)) # Output: [0, 5, 7]

```

7 Grafos

7.1 BFS - Camino más corto

```

1 from collections import deque
2
3 def bfs(graph, start, end):
4     q = deque([start])
5     parent = {start: None}
6
7     while q:
8         u = q.popleft()
9
10        if u == end:
11            break
12
13        for v in graph[u]:
14            if v not in parent: # parent sirve como visitados
15                parent[v] = u
16                q.append(v)
17        if end not in parent:
18            return []
19    path = []
20    while end is not None:
21        path.append(end)
22        end = parent[end]
23
24    return path[::-1]

```

7.2 Dijkstra - Camino menos costoso

Complejidad: $O((N + M) \log N)$ con heapq.

```
1 from heapq import heappush, heappop
2 import math
3
4 def dijkstra(graph, start, end):
5     dist = {u: math.inf for u in graph}
6     parent = {start: None} #Borrar si no se necesita el camino
7
8     dist[start] = 0
9     pq = [(0, start)]
10
11     while pq:
12         d, u = heappop(pq)
13
14         if d > dist[u]:
15             continue
16
17         if u == end:
18             break
19
20         for v, w in graph[u]:
21             if dist[v] > d + w:
22                 dist[v] = d + w
23                 parent[v] = u # Guardar de donde viene
24                 heappush(pq, (dist[v], v))
25
26     # Si no hay camino
27     if dist[end] == math.inf:
28         return math.inf, []
29     # Reconstruir camino
30     camino = []
31     actual = end
32
33     while actual is not None:
34         camino.append(actual)
35         actual = parent[actual]
36
37     camino.reverse()
38
39     return dist[end], camino
40
41 #Ejemplo
42 graph = {
43     1: [(2, 4), (3, 1)],
44     2: [(3, 2), (4, 5), (1, 5)],
45     3: [(4, 8), (2,3)],
46     4: [(3, 2)]
47 }
48 print(dijkstra(graph, 1, 4)) #9 [1, 3, 4]
```

7.3 BFS - Para obtener distancias/profundidad

```
1 from collections import deque
2
3 def bfs(graph, start):
4     dist = {start: 0}
5     queue = deque([start])
6
7     while queue:
8         node = queue.popleft()
9
10        for neighbor in graph[node]:
11            if neighbor not in dist:
12                dist[neighbor] = dist[node] + 1
13                queue.append(neighbor)
14
15    return dist
```

7.4 DFS - Recursivo

Guarda el recorrido en posorder (variable declarada afuera no retorna nada).

Profundidad maxima de recursion < 1000

```
1 def dfs(graph:dict, node:int, visited:set, orden:list):
2     visited.add(node)
3     for neighbor in graph[node]:
4         if neighbor not in visited:
5             dfs(graph, neighbor, visited, orden)
6     orden.append(node)
```

7.5 DFS - Iterativo

No guarda el camino, solo los nodos que se visitaron (se puede hacer que retorne el orden de visita en preorder). No tiene límite de recursion

```
1 def dfs(graph, start):
2     visited = set()
3     stack = [start]
4
5     while stack:
6         node = stack.pop()
7
8         if node in visited:
9             continue
10
11        visited.add(node)
12
13        for neighbor in reversed(graph[node]):
14            if neighbor not in visited:
15                stack.append(neighbor)
16
17    return visited
```

7.6 Kosaraju - Componentes Fuertemente Conexas

```
1 def kosaraju(graph):
2     # Construir grafo transpuesto
3     rev = {u: [] for u in graph}
4
5     for u in graph:
6         for v in graph[u]:
7             rev[v].append(u)
8
9     # 1. DFS para obtener orden de finalizacion
10    visited = set()
11    orden = []
12
13    for u in graph:
14        if u not in visited:
15            dfs(graph, u, visited, orden) #Definir funcion afuera
16
17    # 2. DFS sobre grafo transpuesto
18    # en orden inverso
19    visited = set()
20    sccs = []
21
22    for u in reversed(orden):
23        if u not in visited:
24            comp = set()
25            dfs(rev, u, comp)
26            sccs.append(comp)
27
28    return sccs
```

Uso: Encontrar las componentes fuertemente conexas de un grafo dirigido. Dos vértices pertenecen al mismo SCC si existe un camino de uno al otro y viceversa. $O(V+E)$

7.7 Bipartite Matching - Kuhn

```
1 def dfs(u, graph, visited, match):
2     for v in graph[u]:
3         if v in visited:
4             continue
5
6         visited.add(v)
7
8         # v esta libre o podemos reasignar
9         # al vertice que actualmente lo ocupa
10        if match[v] == -1 or dfs(match[v], graph, visited, match):
11            match[v] = u
12            return True
13
14    return False
15
16 #Grafo dirigido de izq a derecha
17 def bipartite_matching(graph, n_left, n_right):
18     # match[v] = vertice del lado izquierdo
19     # emparejado con v
20     match = [-1] * n_right
21
22     ans = 0
23
24     for u in range(n_left):
25         visited = set()
26
27         if dfs(u, graph, visited, match):
28             ans += 1
29
30     return ans
```

Uso: Encontrar la cantidad maxima de parejas. $O(V \cdot E)$

8 Geometría

8.1 Convex Hull - Monotonic Chain

```
1 def cross(o, a, b):
2     # Producto cruzado:
3     # OA x OB
4     return (a[0] - o[0]) * (b[1] - o[1]) \
5            - (a[1] - o[1]) * (b[0] - o[0])
6
7
8 def convex_hull(points):
9     points = sorted(set(points))
10
11     if len(points) <= 1:
12         return points
13
14     # Parte inferior
15     lower = []
16     for p in points:
17         while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
18             #Si de deja solo < se guardan los puntos colineales
19             lower.pop()
20             lower.append(p)
21
22     # Parte superior
23     upper = []
24     for p in reversed(points):
25         while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
26             upper.pop()
27             upper.append(p)
28
29     # El ultimo punto de cada parte esta repetido
30     return lower[:-1] + upper[:-1]
```

Uso: Encontrar la envolvente convexa de un conjunto de puntos. $O(N \log N)$

8.2 Shoelace - Área de un Polígono

```
1 def polygon_area(points):
2     n = len(points)
3     area = 0
4
5     for i in range(n):
6         x1, y1 = points[i]
7         x2, y2 = points[(i + 1) % n]
8
9         area += x1 * y2 - x2 * y1
10
11     return abs(area) / 2
```

Uso: Calcular el área de un polígono cuyos vértices están dados en orden. $O(N)$

9 Matemáticas

9.1 Aritmética básica

- **Máximo común divisor (MCD) y Mínimo común múltiplo (MCM):**

$$\text{lcm}(a, b) = \frac{|a \cdot b|}{\text{gcd}(a, b)}$$

Uso: simplificar fracciones, comprobar divisibilidad y problemas relacionados con divisores comunes.

9.2 Potencias y logaritmos

- **Logaritmo:**

$$\log_b(x) = y \iff b^y = x$$

Uso: aparece frecuentemente en complejidades, búsqueda binaria, árboles y algoritmos de dividir y conquistar.

- **Cambio de base:**

$$\log_a b = \frac{\log_c b}{\log_c a}$$

Uso: convertir entre bases de logaritmos.

9.3 Sumatorias

- **Suma de los primeros n enteros:**

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Uso: contar pares, operaciones o elementos de estructuras triangulares sin utilizar un ciclo.

- **Suma de cuadrados y cubos:**

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$
$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

- **Suma de una progresión aritmética:**

$$S_n = \frac{n(a_1 + a_n)}{2}, \quad S_n = \frac{n}{2} \cdot [2a_1 + (n-1)d]$$

Uso: secuencias donde la diferencia entre términos es constante. Siendo a el primer y ultimo termino de la secuencia. En caso de no saber el último elemento se usa la segunda formula y d es la diferencia

entre terminos

- **Progresión geométrica:**

$$S_n = a \frac{r^n - 1}{r - 1}, \quad r \neq 1$$

Uso: secuencias donde cada término se obtiene multiplicando por una razón constante.

9.4 Combinatoria

- **Factorial: ($n!$)** Uso: calcular de cuántas formas distintas se pueden ordenar todos los elementos de un conjunto para orden circular es $(n-1)!$

- **Permutaciones con elementos repetidos:**

$$\frac{n!}{n_1!n_2!\dots n_k!}$$

Uso: contar ordenamientos cuando existen elementos repetidos.

- **Permutaciones:**

$$P(n, r) = \frac{n!}{(n-r)!}$$

Uso: seleccionar r elementos de n donde el orden importa.

- **Combinaciones:**

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Uso: seleccionar r elementos de n donde el orden no importa.

- **Propiedad de simetría:**

$$\binom{n}{r} = \binom{n}{n-r}$$

Uso: combinaciones utilizando el menor de r y $n-r$.

9.5 Probabilidad

Buscar librería `itertools` para muchas funciones que generan iteradores

- **Probabilidad básica:**

$$P(A) = \frac{\text{casos favorables}}{\text{casos posibles}}$$

Uso: cuando todos los resultados son igualmente probables.

■ **Complemento:**

$$P(\bar{A}) = 1 - P(A)$$

Uso: calcular “al menos uno” mediante 1– “ninguno”.

■ **Probabilidad de la unión:**

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

■ **Probabilidad condicional:**

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Uso: probabilidad de A sabiendo que B ocurrió.

■ **Eventos independientes:**

$$P(A \cap B) = P(A)P(B)$$

Uso: cuando la ocurrencia de un evento no afecta al otro.

■ **Esperanza matemática:**

$$E[X] = \sum_i x_i P(X = x_i)$$

Uso: calcular el valor esperado de una variable aleatoria.

9.6 Geometría computacional

■ **Producto cruzado en 2D:**

$$\text{cross}(A, B) = x_A y_B - y_A x_B$$

Uso: determinar orientación, áreas y posiciones relativas de vectores.

■ **Orientación de tres puntos:**

$$\text{cross}(B - A, C - A)$$

- > 0 : giro antihorario.
- < 0 : giro horario.
- $= 0$: puntos colineales.

Uso: convex hull, intersección de segmentos y problemas de geometría.

■ **Área de un polígono (Shoelace):**

$$A = \frac{1}{2} \left| \sum_{i=1}^n (x_i y_{i+1} - y_i x_{i+1}) \right|$$

con $(x_{n+1}, y_{n+1}) = (x_1, y_1)$.

Uso: calcular el área de un polígono a partir de sus vértices.

9.7 Sucesiones

■ **Progresión aritmética:**

$$a_n = a_1 + (n - 1)d$$

Uso: encontrar directamente el término n de una secuencia con diferencia constante.

■ **Progresión geométrica:**

$$a_n = a_1 r^{n-1}$$

Uso: encontrar directamente el término n de una secuencia con razón constante.

9.8 Módulo y aritmética modular

■ **Congruencia:**

$$a \equiv b \pmod{m} \iff m \mid (a - b)$$

Uso: problemas donde solo importa el residuo.

■ **Propiedades:**

$$(a + b) \pmod{m} = ((a \pmod{m}) + (b \pmod{m})) \pmod{m}$$

$$(a \cdot b) \pmod{m} = ((a \pmod{m})(b \pmod{m})) \pmod{m}$$

Uso: mantener números pequeños durante cálculos grandes.

```
1 #Potencia modular python:
2 MOD = 10**9 + 7
3 print(pow(2, 100_000, MOD))
```

■ **Pequeño teorema de Fermat:**

$$a^{p-1} \equiv 1 \pmod{p}$$

si p es primo y $p \nmid a$.

Uso: inversos modulares y combinatoria modular.

■ **Inverso modular:**

$$a^{-1} \pmod{m}$$

satisface

$$a \cdot a^{-1} \equiv 1 \pmod{m}.$$

Uso: realizar divisiones bajo módulo.

9.9 Binomio de Newton

$$(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k$$

Uso: expandir potencias y resolver problemas de conteo relacionados con elecciones independientes. Calcula los coeficientes del triangulo de pascal

9.10 Contar dígitos

■ **Número de bits necesarios para representar n :**

$$\lceil \log_2 n \rceil + 1$$

para $n > 0$.

Uso: determinar cuántos bits se necesitan para almacenar un entero (se puede cambiar el logaritmo para calcular numero de dígitos en otras bases).

9.11 Conversiones útiles

■ **Grados a radianes:**

$$\theta_{\text{rad}} = \theta_{\text{deg}} \frac{\pi}{180}$$

■ **Radianes a grados:**

$$\theta_{\text{deg}} = \theta_{\text{rad}} \frac{180}{\pi}$$

■ **Velocidad:**

$$v = \frac{d}{t}$$

Uso: problemas de movimiento y distancia/tiempo.

■ **Densidad:**

$$\rho = \frac{m}{V}$$

Uso: problemas físicos que relacionan masa y volu-

men.

9.12 Fórmulas de crecimiento

■ Crecimiento cuadrático:

$$f(n) = an^2 + bn + c$$

Uso: problemas donde aparecen pares de elementos o combinaciones de dos dimensiones.

■ Crecimiento exponencial:

$$f(n) = a^n$$

Uso: enumeración de subconjuntos, estados o posibilidades.

■ Relación logarítmica:

$$2^k \leq n \iff k \leq \log_2 n$$

Uso: estimar cuántas veces puede duplicarse un valor antes de superar n .

9.13 Geometría

■ Distancia Manhattan:

$$d = |x_2 - x_1| + |y_2 - y_1|$$

Uso: movimiento en una cuadrícula cuando solo se puede avanzar horizontal o verticalmente.

■ Distancia al cuadrado:

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

Uso: comparar distancias sin calcular la raíz cuadrada.

■ Triángulo

Base b y altura h :

$$A = \frac{bh}{2}$$

Para lados a, b, c , semiperímetro

$$s = \frac{a + b + c}{2}$$

y fórmula de Herón:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

Uso: calcular el área de un triángulo cuando se conocen la base y altura, o sus tres lados.

Perímetro:

$$P = a + b + c$$

■ Volumen de una esfera:

$$V = \frac{4}{3}\pi r^3$$

■ Círculo

Radio r :

$$A = \pi r^2 \quad C = 2\pi r$$

Uso: área y perímetro (circunferencia) de un círculo.

■ Semicírculo

Radio r :

$$A = \frac{\pi r^2}{2}$$
$$P = \pi r + 2r$$

Uso: regiones formadas por medio círculo, incluyendo el diámetro en el perímetro.

■ Trapecio

Bases B, b y altura h :

$$A = \frac{(B + b)h}{2}$$

Si sus lados son a, b, c, d :

$$P = a + b + c + d$$

Uso: áreas de cuadriláteros con al menos un par de lados paralelos.

■ Paralelogramo

Base b y altura h :

$$A = bh$$

Si los lados miden a y b :

$$P = 2(a + b)$$

Uso: regiones con lados opuestos paralelos.

■ Rombo

Diagonales d_1, d_2 :

$$A = \frac{d_1 d_2}{2}$$

Si el lado mide a :

$$P = 4a$$

Uso: cuadriláteros con cuatro lados iguales.

■ Elipse

Semiejes a y b :

$$A = \pi ab$$

Uso: principalmente para calcular el área de una elipse. El perímetro no tiene una fórmula elemental simple.

■ Polígono regular

n lados de longitud a y apotema r :

$$P = na$$

$$A = \frac{Pr}{2}$$

También:

$$A = \frac{na^2}{4} \cot\left(\frac{\pi}{n}\right)$$

Uso: áreas y perímetros de polígonos regulares.

■ Sector circular

Radio r y ángulo θ en radianes:

$$A = \frac{1}{2}r^2\theta$$

$$L = r\theta$$

donde L es la longitud del arco.

Uso: regiones que corresponden a una fracción de un círculo.